ASHESI UNIVERSITY

ICS 534 CLOUD COMPUTING | PROSIT 1 → 2 | WEEK 5 LAB REPORT

HealthConnect Ghana

A Segmented VPC, a Load-Balanced Service & a Multi-Country Data Residency Plan

Group 3 · Team Highlanders

Joseph Etse · Eugene Sewor · Elise Kennedy-Angbo · Perfect Avugla

Ashesi University · Submitted October 2026



Our Lab in Numbers

6

Subnets across 2 Availability Zones: public, private-app and private-data

0

Routes from the database subnets to the internet: their route table has only the local route

2

Copies of our API behind the load balancer, so one can fail and the other keeps answering

74

AgroConnect resources torn down with one terraform destroy before HealthConnect began

$3

Monthly cost of our NAT instance, against about $32 for a managed NAT Gateway

2

Countries in our residency design, each with its own database

1. Lab Report: Our Network and What Changed

WHY THIS NETWORK IS HEALTHCONNECT'S

Lab 5 finishes PROSIT 1, and Lab 6 carries this VPC design into HealthConnect. We built the Week 5 network for HealthConnect using our AgroConnect Terraform modules, creating the core network for all upcoming labs.

HOW THE NETWORK IS SEGMENTED

The VPC (10.30.0.0/16) spans two Availability Zones across three clear tiers: public subnets for the load balancer and NAT instance, private app subnets for API servers, and private data subnets for PostgreSQL. Distinct route tables enforce boundaries: the public table routes to the internet gateway, the app table routes outbound traffic through the NAT instance so servers can download updates without being publicly accessible, and the data table stays strictly local with zero internet access.

THE DATABASE AND THE LOAD BALANCER

The database uses RDS PostgreSQL 16 with encryption at rest and public access disabled, restricting port 5432 connections to the app servers. An Auto Scaling group maintains two patient-record-service instances behind an Application Load Balancer that checks /health every 15 seconds. When we stopped one instance, the load balancer removed it within 18 seconds while the second instance handled traffic. We are investigating why a brief set of request timeouts occurred during the transition as our first item for Lab 6.

WHAT CHANGED FROM PROSIT 1

HealthConnect expands on PROSIT 1 by running two API instances instead of one to demonstrate active load balancing. It is a dedicated codebase and network tailored for sensitive health data. The Terraform code was deployed in our team account (af-south-1) and successfully re-applied in a second account (eu-north-1) to verify reproducibility. HTTPS configuration will follow in Lab 6 once domain names are assigned.

2. Consolidated Architecture

Client → load balancer → application tier → data tier → external services, with the public and private subnet boundaries marked.

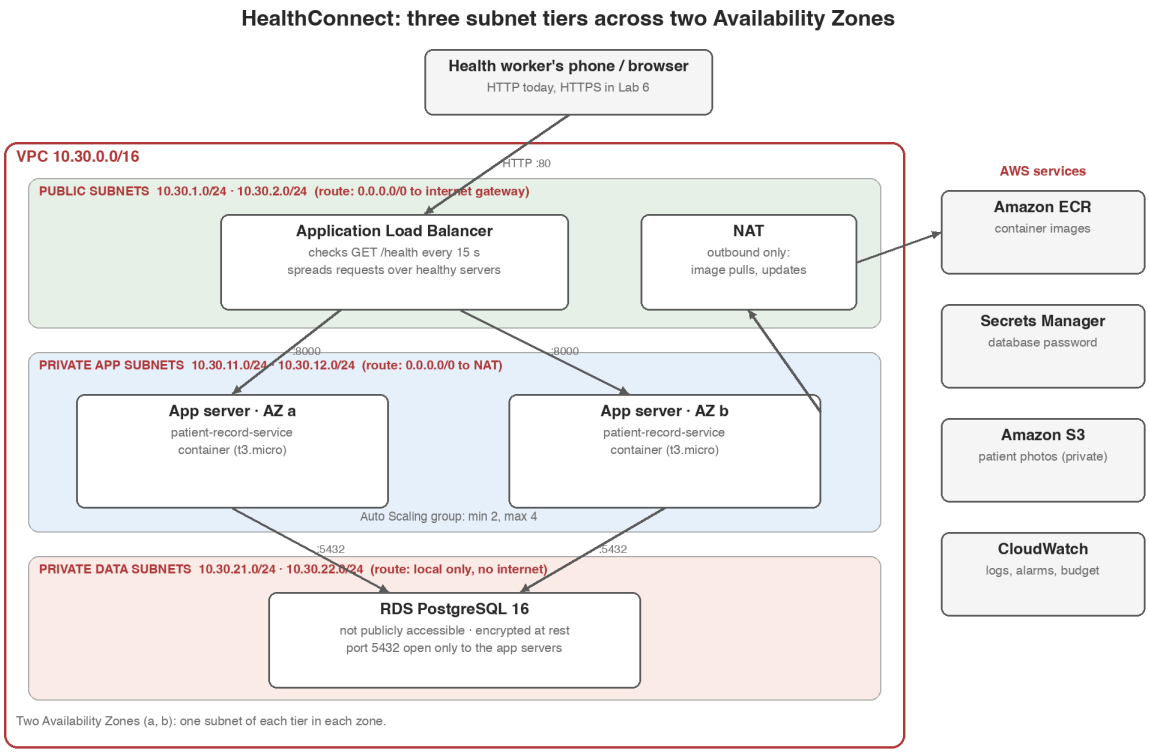


Figure 1. HealthConnect architecture: three subnet tiers across two Availability Zones.

3. The VPC (Part A)

Everything below is Terraform (infra/ in our repository), applied with one command. Each tier has one subnet in each Availability Zone, so losing a zone leaves every tier running. One difference in the second account (eu-north-1), where the screenshots were taken: AWS had no capacity that night for the small ARM instance our NAT instance runs on, so that copy uses a managed NAT Gateway instead. The boundary is the same; only the NAT product differs.

Setting	Value
VPC	healthconnect-dev-vpc, 10.30.0.0/16 (AgroConnect used 10.20.0.0/16, so the two never overlap)
Availability Zones	2
Public subnets	10.30.1.0/24, 10.30.2.0/24: load balancer and NAT instance

Setting	Value
Private app subnets	10.30.11.0/24, 10.30.12.0/24: the API servers
Private data subnets	10.30.21.0/24, 10.30.22.0/24: PostgreSQL only
Public route table	0.0.0.0/0 → internet gateway
App route table	0.0.0.0/0 → NAT instance (fck-nat, t4g.nano) in the team account; a managed NAT Gateway in the eu-north-1 copy
Data route table	10.30.0.0/16 → local, and nothing else

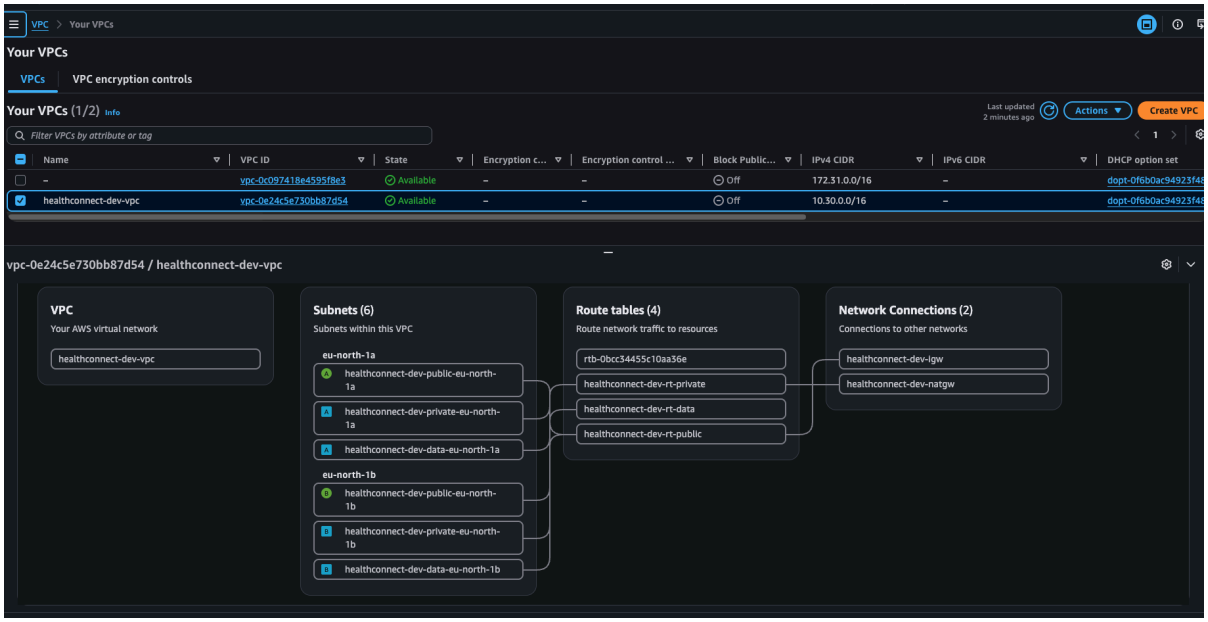


Figure 2. VPC resource map: six subnets in two Availability Zones and their route tables.

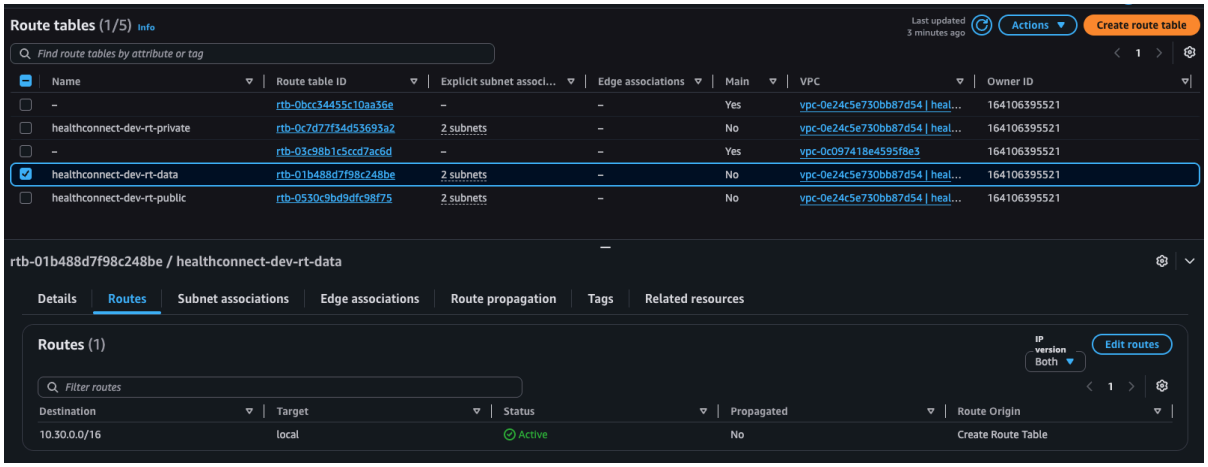


Figure 3. The data subnets' route table: no route to an internet gateway or NAT.

Route-table check (verify_subnet_isolation), captured in our team account on 22 October:

```
== healthconnect-dev-rt-public
| 10.30.0.0/16 | local                | active |
| 0.0.0.0/0    | igw-03cc4cd848cfab955 | active |
== healthconnect-dev-rt-private
| 10.30.0.0/16 | local                | active |
| 0.0.0.0/0    | eni-06a763513ecb7322b | active |   (NAT instance)
== healthconnect-dev-rt-data
| 10.30.0.0/16 | local                | active |   (no 0.0.0.0/0)
```

Figure 4. Public → internet gateway; private app → NAT instance; data → local only.

4. Private Database and Load Balancer (Part B)

Setting	Value
Database	healthconnect-dev-postgres: RDS PostgreSQL 16, db.t4g.micro, 20 GB
Publicly accessible	No
Subnets	The two private data subnets only
Who can connect	Port 5432, only from the app servers' security group
Encryption at rest	On
Password	Managed by RDS in AWS Secrets Manager; never in code or files
Load balancer	healthconnect-dev-alb: internet-facing, in the public subnets
Health check	GET /health every 15 s; healthy after 2 passes, unhealthy after 5 failures
App servers	Auto Scaling group: minimum 2, maximum 4 (t3.micro), in the private app subnets
Service	patient-record-service container from our ECR registry; /health names the instance that answered

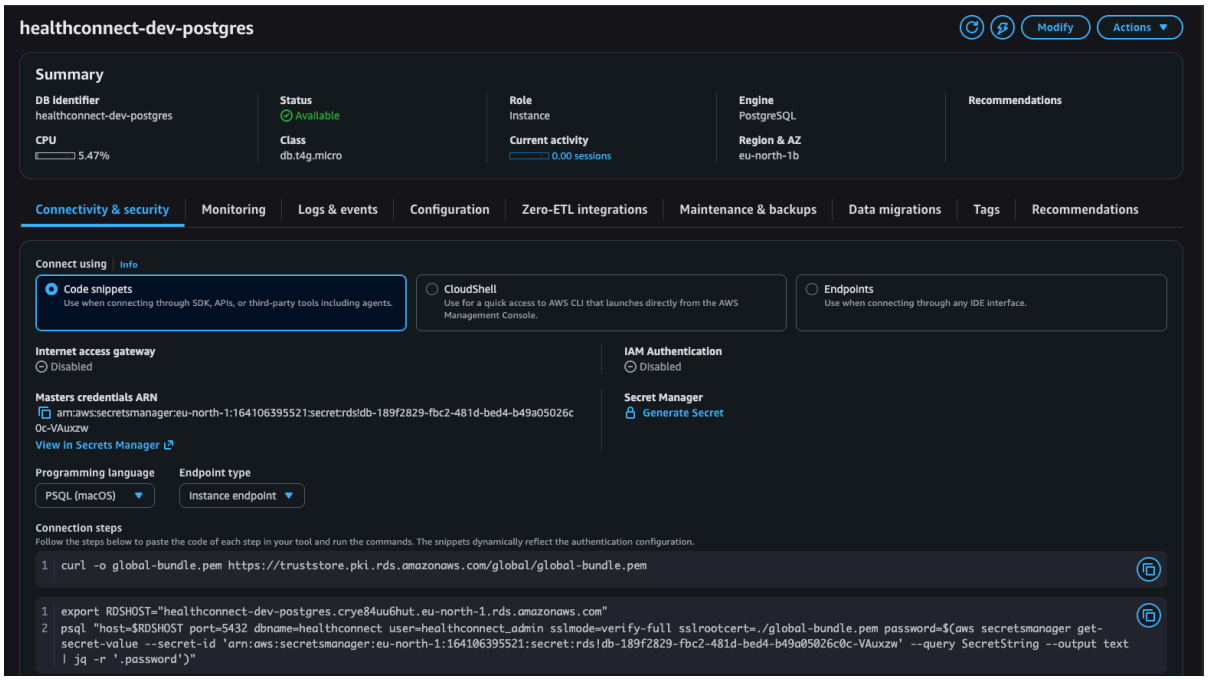


Figure 5. The database: private data subnets, not publicly accessible.

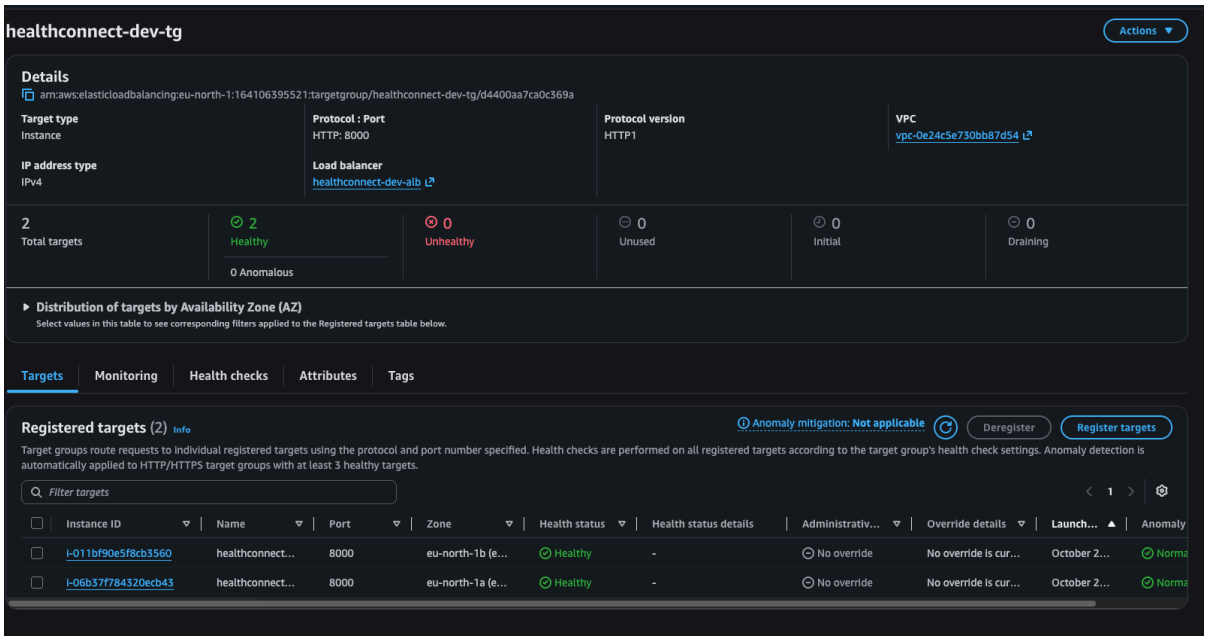


Figure 6. Both instances healthy behind the load balancer.

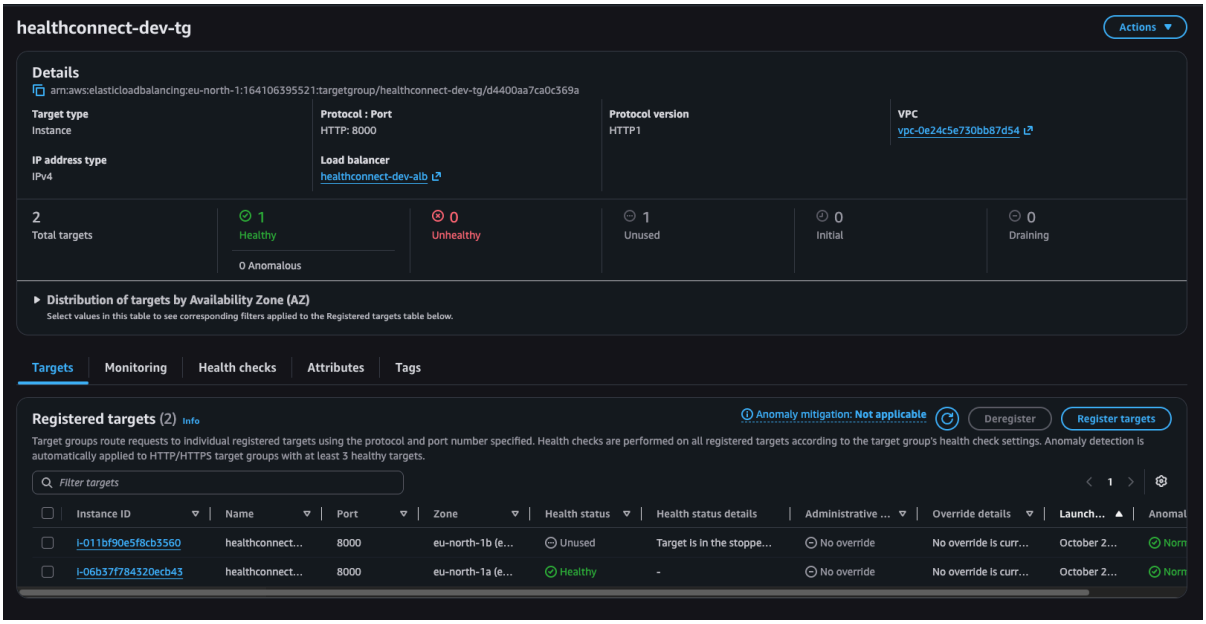


Figure 7. One instance stopped: the health check marks it unused ("Target is in the stopped state") and 1 healthy target remains.

Failover test (lab5_part2_failover.sh, 22 October 23:17 UTC): 10 requests before and after stopping one instance.

```
=== BEFORE: 10 requests ===
1: {"instance":"i-011bf90e5f8cb3560"} HTTP 200    2: {"instance":"i-06b37f784320ecb43"} HTTP 200
... both instances answer ...  => 10/10 succeeded
Stopping i-011bf90e5f8cb3560 ... 23:19:14 i-011bf90e5f8cb3560: unused (Target.InvalidState)
=== AFTER: 10 requests ===
1: {"instance":"i-06b37f784320ecb43"} HTTP 200    2: HTTP 000 (timed out after 5 s)
3: {"instance":"i-06b37f784320ecb43"} HTTP 200    4: HTTP 000
... => 5/10 succeeded; every success came from the surviving instance
```

Figure 8. Failover: the health check removed the stopped instance within about 18 seconds. A second run, waiting 30 s and allowing 15 s per request, still saw 504 Gateway Time-outs, so this is not just in-flight requests.

WHAT THIS TELLS US

The health checks successfully detected the stopped server and redirected active requests to the remaining instance. However, temporary client-side timeouts showed that failover is not yet fully seamless. Since cross-zone load balancing is already enabled, we will review deregistration delays and target group draining settings before deploying live services.

5. Multi-Country Data Residency (Part C)

COUNTRIES AND SHARDING

We designed the system for deployment in Ghana and Kenya. Each patient record is tagged with a country code (GH or KE) at registration based on the health facility location. To strictly separate data, each country operates its own PostgreSQL database containing its local patient records, consents, readings, appointments, and audit logs. The API detects the user's country context and routes queries to the corresponding database without cross-country joins. For cross-border analytics, a nightly batch job exports aggregated summaries, automatically omitting any counts lower than 5 to protect privacy.

Country	Law, and one way our design respects it
Ghana	Data Protection Act, 2012 (Act 843). Health data is special personal data under the Act, processed only with consent: we record consent per purpose before any reading is stored, and Ghanaian records never leave the Ghana database; only anonymised totals do.
Kenya	Data Protection Act, 2019, with the Data Protection (General) Regulations, 2021. Health data is sensitive personal data, and processing for primary and secondary health care must, as we read the Regulations, use a server or data centre in Kenya: the Kenya database must sit in Kenya.

WHERE EACH DATABASE RUNS

AWS has no region in Ghana or Kenya. The Ghana database runs in af-south-1 (Cape Town), the lowest-latency region we measured from Ghana (74 ms). The Kenya database cannot simply be "a second AWS region": it needs an in-country data centre or provider. That is a real cost and architecture consequence of the law, not a configuration flag.

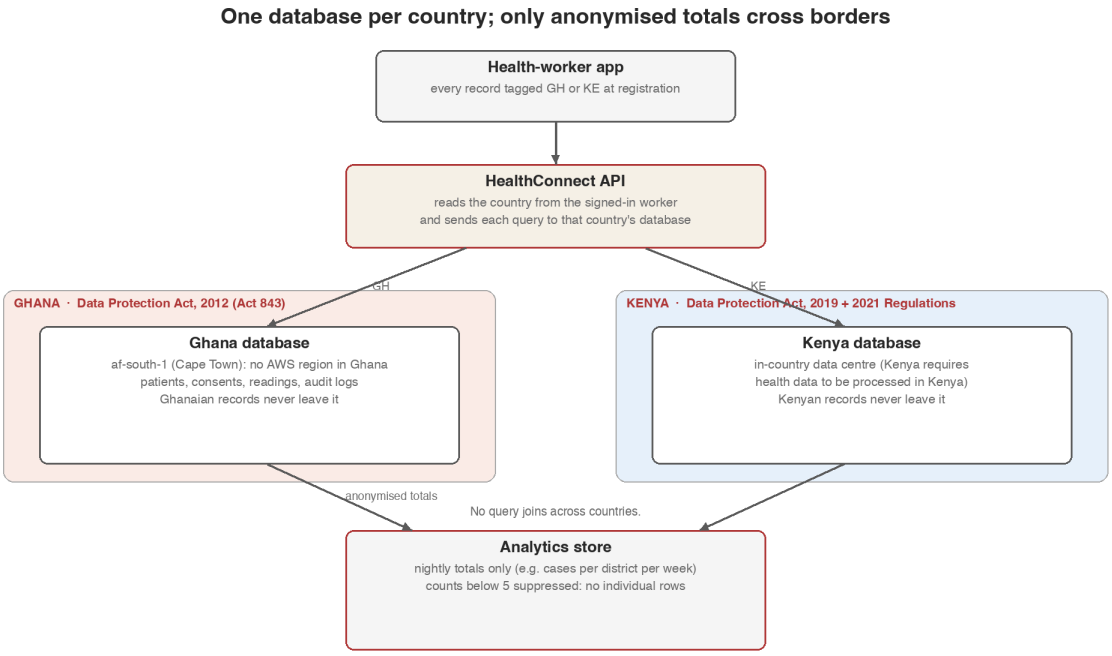


Figure 9. One database per country; only anonymised totals cross borders.

WHAT STILL NEEDS REAL LEGAL REVIEW

Prior to production deployment, formal legal review will address: (1) compliance of storing Ghanaian data in South Africa under Act 843; (2) registration requirements with the Data Protection Commissions in Ghana and Kenya; (3) exact in-country server requirements for Kenya; (4) data transfer protocols when patients move across borders; (5) consent handling for minors; (6) record retention limits for withdrawn users; and (7) legal validation of suppressing low count values for anonymisation.

6. Decommissioning AgroConnect (Part D)

Resource	State, and why
AgroConnect stack (VPC, ALB, NAT instance, Auto Scaling group, RDS, S3, Amplify)	Destroyed: all 74 resources removed with terraform destroy on 15 October, after PROSIT 1 was submitted
Terraform state bucket	Kept: shared with HealthConnect under a separate key; costs cents a month

Resource	State, and why
Week 4 Aurora database and media bucket (eu-north-1)	Deleted: confirmed by our audit script (no Aurora cluster or media bucket remains)
NAT Gateways	One, temporary: only in the eu-north-1 copy, because the NAT instance had no capacity there. It bills hourly, so it is deleted with that copy right after grading. The team account uses the NAT instance.
HealthConnect stack (af-south-1, team account)	Running: needed for Labs 5–7 and the PROSIT 2 demo
HealthConnect copy (eu-north-1, second account)	Running tonight for this report's evidence; destroyed after grading
Azure and GCP accounts (Week 1)	Not re-checked for this report; on our list to confirm zero running resources
test-bucket-agroconnect (S3, second account)	Found by the audit: a test bucket from 12 October. To delete after grading



Figure 10. Month-to-date spend $77.10, forecast $95.09 for October. Our budget alarm emails the team at $50 forecast and $100 actual, so October will trip the warning: the main reason we tear environments down promptly.

7. Offline-First PWA (from Week 4)

The offline-capable PWA was built and demonstrated in our Week 4 lab and is unchanged. The live AgroConnect site was retired with the rest of AgroConnect on 15 October; the code remains in our PROSIT 1 repository, and HealthConnect's health-worker app will be built from it.

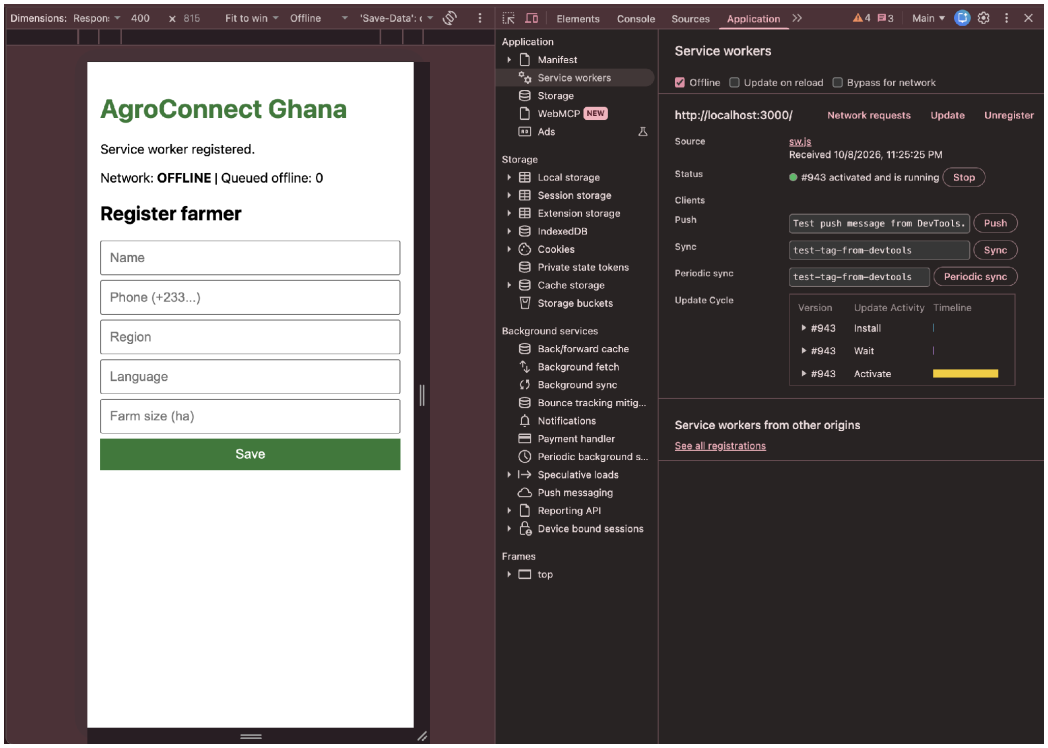


Figure 11. DevTools Offline checked: the service worker is running and the page still renders with "Network: OFFLINE" (Week 4).

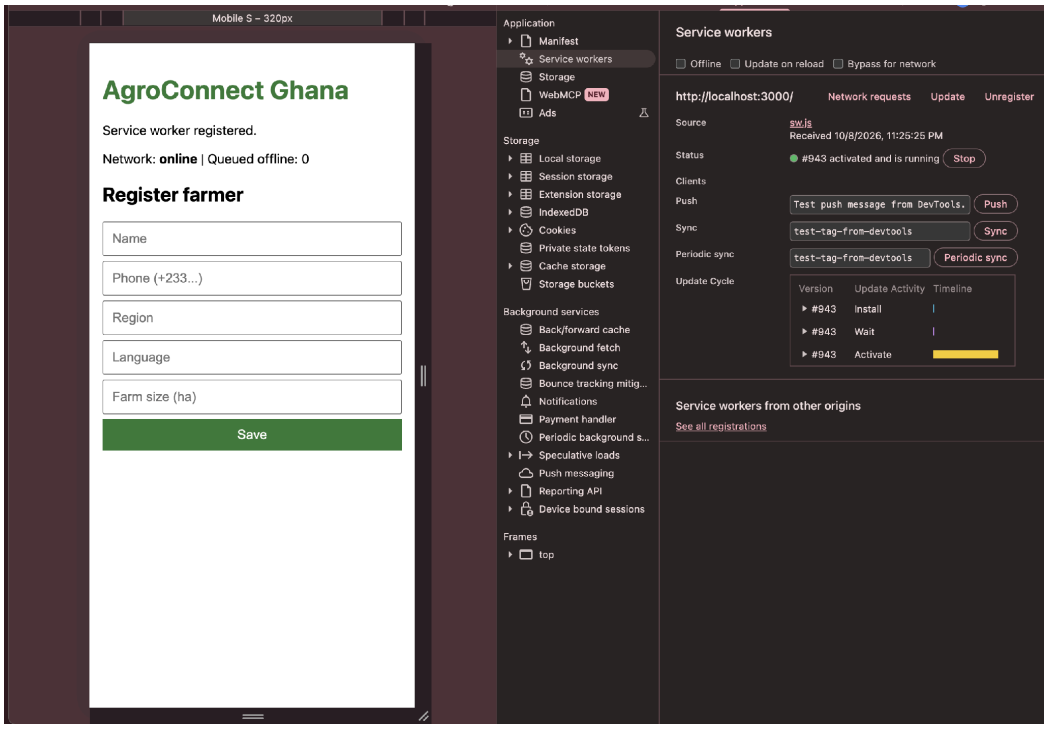


Figure 12. Back online: the same service worker is still running and the offline queue has emptied (Week 4).

8. Retrospective

If we started AgroConnect again, we would build private subnets in Week 2 instead of fixing them in Week 5: our Week 4 database had a public endpoint guarded only by an IP allow-list, the riskiest thing we built. We would keep every credential in Secrets Manager from day one; a database password ended up in a plain-text file in our shared Drive. We would run two API instances from the start, because a design that can fail over is not the same as one that has been seen to. We would snapshot the database before tearing an environment down; ours was deleted outright, acceptable only because the data was synthetic. And we would talk to real farmers and extension agents early: our users' needs are still assumptions taken from the brief.

CONTAINER REGISTRY

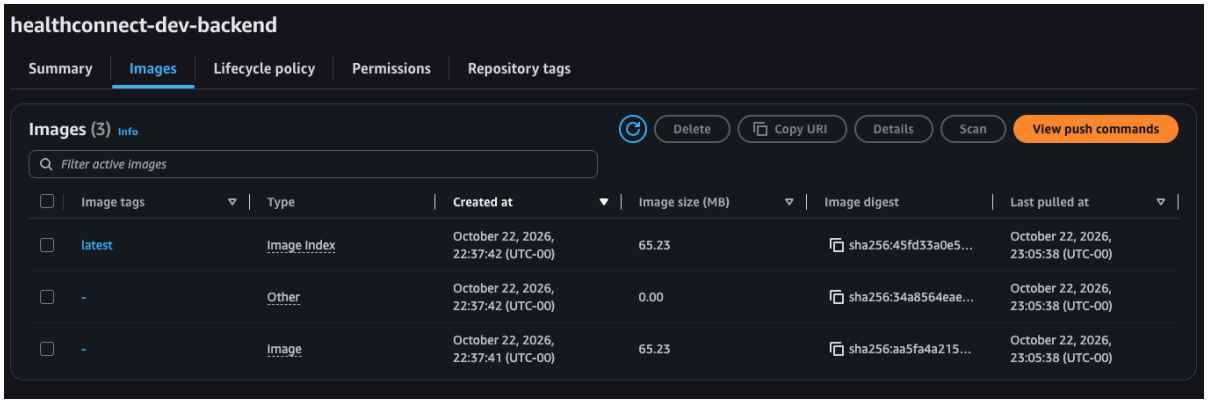


Figure 13. Amazon ECR: the healthconnect-dev-backend repository holds our patient-record-service image (tag latest, 65 MB).

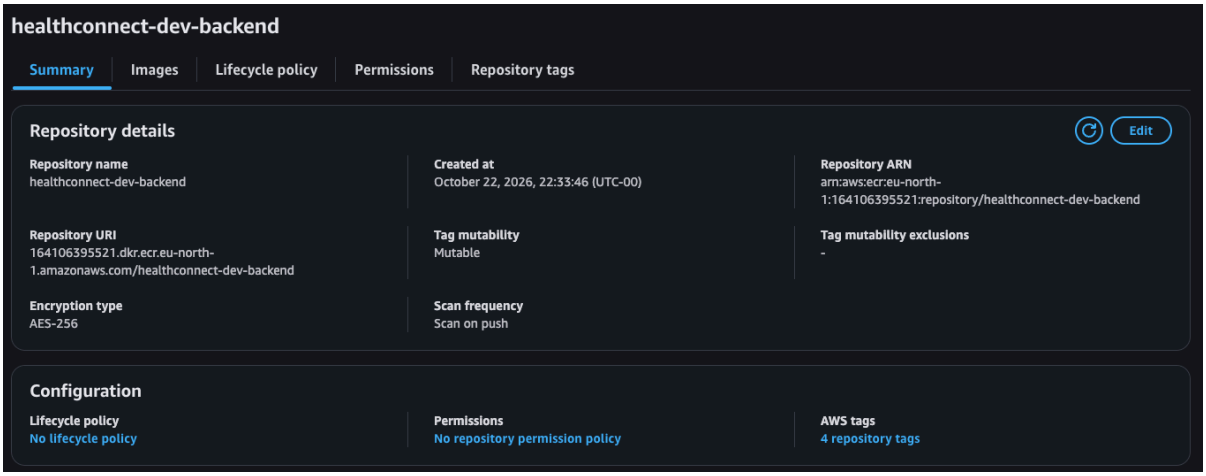


Figure 14. Repository details: encrypted (AES-256) and scanned for vulnerabilities on every push.

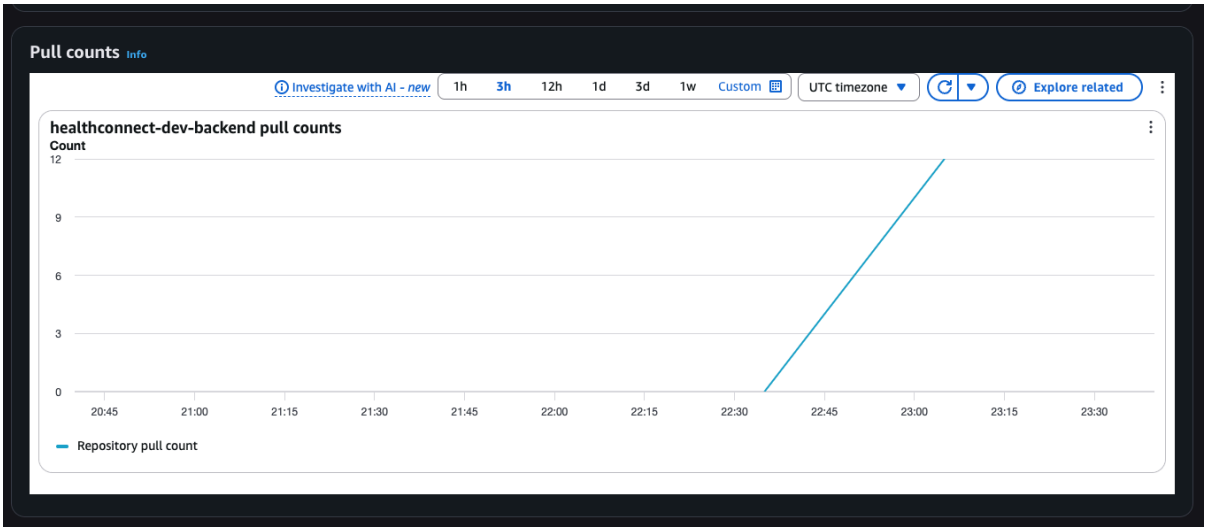


Figure 15. Pull count: the app servers pulled the image when they booted (22:35–23:05).

9. Links

What	Where
Code and infrastructure	github.com/ASU-MICS-2028/cc-group-3-prosit-2 (infra/, backend/)
Container registry	Amazon ECR: healthconnect-dev-backend
Load-balanced service	http://healthconnect-dev-alb-123985859.eu-north-1.elb.amazonaws.com/health (second account; HTTP until HTTPS in Lab 6)

What	Where
Database	healthconnect-dev-postgres: private by design, so there is no public link
Offline PWA (Week 4)	github.com/ASU-MICS-2028/cc-group-3-prosit-1 (frontend/)

10. Deliverables Checklist

Rubric criterion	Weight	Where to find the evidence
VPC correctly segmented, database not publicly reachable	20%	Sections 3 and 4, Figures 2–5
Load balancer with demonstrated health-check failover	15%	Section 4, Figures 6–8
Data residency design specific and legally grounded	20%	Section 5, Figure 9
Offline-first PWA functioning end-to-end (Week 4)	15%	Section 7, Figures 11–12
Retrospective honesty and depth	20%	Section 8
Decommissioning checklist with honest spend	10%	Section 6, Figure 10